Fast Fully Oblivious Sampling

Michael Shell
School of Electrical and Computer Engineering
Georgia Institute of Technology
Atlanta, Georgia 30332–0250
Email: http://www.michaelshell.org/contact.html

Homer Simpson
Twentieth Century Fox
Springfield, USA
Email: homer@thesimpsons.com

James Kirk
and Montgomery Scott
Starfleet Academy
San Francisco, California 96678-2391
Telephone: (800) 555–1212
Fax: (888) 555–1212

***Abstract*—The abstract goes here.**

1. Introduction

Oblivious algorithms [1], [2], [3], [4], [5], [6] enable privacy-preserving computation over sensitive data by hiding data-dependent access patterns from untrusted observers. They have been widely applied to privacy-preserving systems, including machine learning [2], [7], graph processing [8], distributed data analytics [9], and private statistics [10]. For example, Weave [9] enables large-scale privacy-preserving data analytics within trusted execution environments (TEEs), such as Intel SGX [11]. Specifically, it conceals data-dependent communication and memory-access patterns during distributed computation, preventing an untrusted cloud provider from inferring sensitive information from how records are processed.

Oblivious sampling [10], [12] plays an important role in privacy-preserving data analysis by enabling computations on randomly selected subsets without revealing which records are sampled. Such functionality supports secure approximate query processing [13], privacy-preserving statistical estimation [10], [14], and mini-batch machine learning [15], [16], where operating on small subsets can substantially reduce computational costs. Beyond efficiency, concealing sample membership is also important for differential privacy. Under appropriate sampling strategies, sampling can amplify the privacy guarantees of differentially private computations [14], [17], [18].

Despite its importance, oblivious sampling remains expensive when multiple independent samples are required. Generating each sample separately incurs repeated dataset-scale processing, motivating *batch sampling*. PSQF [12] proposed a TEE-oriented batch sampling algorithm that generates multiple samples without replacement using two oblivious shuffles and a replication scan. MASQUE [13] further explores secure batch sampling in the MPC setting. However, existing TEE-based approaches still rely on costly global oblivious shuffles, leaving substantial room for improving the concrete efficiency of oblivious batch sampling.

In this paper, we explore the following question: *Can we achieve concretely efficient, fully oblivious batch sampling that generates independent samples without replacement while minimizing redundant data movement?*

2. Preliminaries

2.1. Batch Sampling Without Replacement

Sampling without replacement (SWO) selects m distinct records uniformly at random from a dataset of n records [19]. Batch SWO independently generates k such samples from the same dataset [12], [13]. Thus, each record appears at most once within a sample, but may appear in multiple samples.

Definition 1. *Let $D = (d_0, \ldots, d_{n-1})$ be a dataset of n indexed records, and let $1 \le m \le n$ and $k \ge 1$. Define $[n] = \{0, \ldots, n-1\}$. A randomized algorithm $\mathcal{A}$ realizes batch SWO if, for every such dataset D, $\mathcal{A}(D, m, k)$ outputs k samples $S = (S_0, \ldots, S_{k-1})$ satisfying the following conditions.*

Each sample S_j consists of m records from D with distinct input indices. Let $I_j \subseteq [n]$ denote the indices of the records selected into S_j. The index sets $I_0, \ldots, I_{k-1}$ are mutually independent, and for every $I \subseteq [n]$ with $|I| = m$,

$$\Pr[I_j = I] = \frac{1}{\binom{n}{m}}, \qquad 0 \le j < k,$$

where the probability is over the randomness of $\mathcal{A}$. The order of records within each sample is unspecified.

2.2. Full Obliviousness

In TEEs, protecting data contents does not necessarily conceal how data are accessed or processed. An adversary may observe accesses to external memory, infer accesses to protected memory through side channels, or observe the executed instruction sequence. Following prior work [4], [5], we adopt *full obliviousness*, which requires both memory-access and instruction traces to be simulatable using only public information.

Definition 2. *Let $\mathcal{A}$ be a probabilistic polynomial-time algorithm with public input P, secret input x, and security parameter λ, where P includes the sizes of secret inputs and other public execution parameters. Let E denote an execution of $\mathcal{A}(1^\lambda, P, x)$, and let $O(E)$ denote its output. Let $M(E)$ denote the memory-access trace, recording the instruction step, memory address, and read/write operation of each access. Let $I(E)$ denote the sequence of executed instruction identifiers, and define the execution trace as $T(E) = (M(E), I(E))$.*

$\mathcal{A}$ is fully oblivious *with respect to x if there exists a probabilistic polynomial-time simulator* Sim *such that, for every valid (P, x),*

$$\big(O(E), T(E)\big) \approx_{\mathrm{c}} \big(O(E), \mathsf{Sim}(1^\lambda, P)\big),$$

where $\approx_{\mathrm{c}}$ denotes computational indistinguishability as a function of λ. The simulator uses independent randomness and receives neither the secret input nor the output.

For batch SWO, the public parameters include n, m, k, and the record size, while both the input data and the sampling decisions are sensitive. An oblivious batch-SWO algorithm must satisfy the sampling requirements in Definition 1 and the full obliviousness requirement in Definition 2. To protect the sampling decisions, we interpret $O(E)$ in the security definition as the logical sampling outcome consisting of indexed records, even if their original indices are not explicitly returned by the implementation. Consequently, the execution trace must not reveal which input records are selected into each sample or shared across different samples.

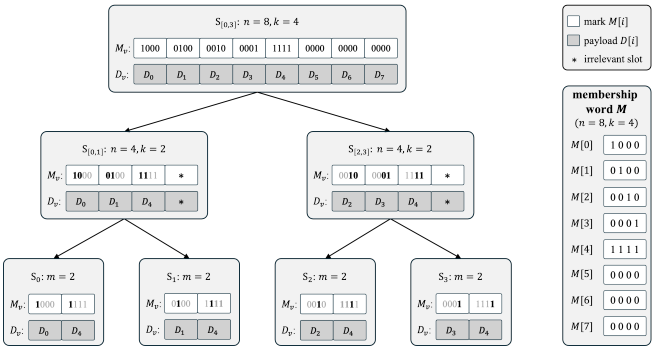


Figure 1. Hierarchical binary-tree sampling framework.

2.3. Oblivious Compaction

3. FFOS-C: Compaction-Based Fast Fully Oblivious Sampling

3.1. Design Overview

Our sampling algorithm is built around the binary sampling tree illustrated in Figure 1. The root represents all k samples, each internal node represents a group of samples and divides it into two approximately equal subgroups, and each leaf corresponds to one final sample. Each node maintains a candidate array containing the records that may still be needed by at least one sample in its subtree. As the traversal proceeds from the root toward the leaves, sample groups are progressively separated and the corresponding candidate arrays become smaller.

To determine how records propagate through the tree, we associate each input record $D[i]$ with a k-bit membership word $M[i]$. Bit j of $M[i]$ indicates whether $D[i]$ belongs to sample S_j. Hence, for a node representing a sample group G_v, a record is relevant to that node whenever at least one membership bit corresponding to G_v is set. At an internal node, a record may therefore be needed by the left child, the right child, both children, or neither child. In particular, samples are independent but need not be disjoint, so a record selected by samples in both child groups must propagate to both subtrees.

The tree also bounds the amount of data that must be retained at each node. If node v represents k_v samples, these samples contain at most mk_v distinct input records in total. Thus, if the parent candidate array has length n_{par}, the public

Algorithm 1 FFOS-C(D, n, m, k)

Input: Data array D of public length n, public sample size m and number of samples k;
Output: Sample list $\mathcal{S}$ of k independent uniform samples;

Offline phase
$M \leftarrow$ MARKSAMPLES(n, m, k)
$C \leftarrow$ FFOSCCTRBITS(M, n, m, k)
Online phase
$(\mathcal{S}, p) \leftarrow$ FFOSCAPPLY$(D, C, n, m, k, 0)$
return $\mathcal{S}$

Algorithm 2 MARKSAMPLES(n, m, k)

Input: Public length n, sample size m, and number of samples k;
Output: Membership array M of length n;

Create a length-n array M initialized to 0.
Create a length-k array R initialized to m.
for $i \leftarrow 0 \dots n-1$ **do**
  for $j \leftarrow 0 \dots k-1$ **do**
    Generate $\rho \in [0, 1)$ uniformly at random.
    $u \leftarrow [\![\rho < R[j]/(n-i)]\!]$ ▷ *Selection bit*
    $M[i] \leftarrow M[i] \mid (u \ll j)$
    $R[j] \leftarrow R[j] - u$ ▷ *Update quota*
return M

capacity of node v is $n_v = \min\{n_{\mathsf{par}}, mk_v\}$. Only the first n_v positions need to be retained for the subtree. If fewer than n_v records are actually relevant, the retained prefix may additionally contain irrelevant records. The retained length therefore depends only on public parameters and not on the actual overlap among the descendant samples. At a leaf, $k_v = 1$ and the candidate-array capacity becomes m, so the m relevant records form the final sample.

Figure 1 also illustrates a second key idea: the same sampling tree is used in both the offline and online phases. Sampling membership depends only on public input positions and fresh randomness, rather than on the payload values. The offline phase therefore operates only on lightweight membership information, propagating membership words through the tree and deriving the secret controls required at each node. The online phase later follows the same public tree structure and replays these controls on the actual records. In this way, sampling-dependent decisions are separated from payload processing: the offline phase determines how records should move, whereas the online phase only executes the predetermined oblivious operations.

The binary-tree framework specifies which candidate records each child node must receive, but does not prescribe how the two child candidate arrays are obliviously materialized. FFOS-C (Sec. 3) realizes this node operation using oblivious compaction. FFOS-FR (Sec. 4), introduced in Section 4, uses the same sampling tree but realizes suitable node splits with oblivious fork routing. The remainder of this section focuses on the compaction-based realization.

Notation. Arrays use zero-based indexing. We write $A[i]$ for the element of A at index i, $|A|$ for its length, and $A[a : b)$ for the half-open subarray $(A[a], \dots, A[b-1])$. We write $[\![Q]\!]$ for the obliviously computed indicator of predicate Q, equal to 1 if Q holds and 0 otherwise. The operators & and | denote bitwise AND and OR, respectively, and $\ll$ and $\gg$ denote left and right bit shifts. We use $\|$ for ordered concatenation. These primitives can be implemented efficiently and obliviously using boolean arithmetic or conditional instructions such as CMOVZ.

3.2. Offline Control Generation

In FFOS-C, each child candidate array in the sampling tree is materialized using oblivious compaction. A compaction therefore requires, for every position in the current candidate array, a secret bit indicating whether the corresponding record is needed by at least one sample in the child subtree. Since these decisions depend only on sample membership rather than on the payload values, they can be prepared before the data records are available. The offline phase first generates the membership array M using MARKSAMPLES (Algorithm 2), and then transforms these memberships into the compaction controls required throughout the sampling tree using FFOSCCTRBITS (Algorithm 3).

MARKSAMPLES independently generates a uniform size-m subset of the n input positions for each sample. For sample j, it maintains a secret remaining quota $R[j]$ and selects position i with probability $R[j]/(n-i)$. This sequential rule produces exactly m positions uniformly without replacement. Fresh independent randomness across samples makes the corresponding columns of M independent, while still allowing different samples to overlap. Thus, M jointly captures the sampling decisions for all k samples with respect to the original input array.

The original membership array, however, is not sufficient to derive all controls directly. At the root, its membership words immediately determine which input positions are needed by each child sample group. After this first split, however, each child operates on a candidate array whose records have been reordered by compaction and whose public length may have been reduced. Controls for deeper nodes must therefore be derived with respect to the local record order produced by their ancestors, rather than the original input order.

To achieve this, FFOSCCTRBITS propagates the membership information through the same tree transformations that the data records will later undergo online. For every node v, the resulting membership array M_v is kept positionally aligned with the corresponding future candidate array D_v: $M_v[i]$ describes the record that will occupy $D_v[i]$. The membership bits associated with each child determine its compaction flags, and applying these flags to the membership words produces the membership state for that child. Because the same flags induce the same compaction permutation independently of the array contents, replaying those controls on the data records preserves this correspondence at every level. Retaining the public prefix $n_v = \min\{n, mk_v\}$

Algorithm 3 FFOSCCTRBITS(M, n, m, k)

Input: Membership array M of length n, public sample size m and number of samples k;
Output: Secret one-bit compaction-control array C;

if $k = 1$ **then return** $[\,]$
$k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$ ▷ *Split samples*
$\mu_l \leftarrow (1 \ll k_l) - 1$, $\mu_r \leftarrow (1 \ll k_r) - 1$
Create bit arrays C_l, C_r, each of length n.
for $i \leftarrow 0 \dots n-1$ **do**
    $C_l[i] \leftarrow [\![(M[i] \,\&\, \mu_l) \neq 0]\!]$
    $C_r[i] \leftarrow [\![((M[i] \gg k_l) \,\&\, \mu_r) \neq 0]\!]$
$n_l \leftarrow \min(n, mk_l)$, $n_r \leftarrow \min(n, mk_r)$
Copy M into independent arrays M_l and M_r.
OCOMPACT(M_l, C_l)
OCOMPACT(M_r, C_r)
for $i \leftarrow 0 \dots n_l - 1$ **do**
    $M_l[i] \leftarrow M_l[i] \,\&\, \mu_l$ ▷ *Keep left bits*
for $i \leftarrow 0 \dots n_r - 1$ **do**
    $M_r[i] \leftarrow (M_r[i] \gg k_l) \,\&\, \mu_r$ ▷ *Reindex right bits*
$M_l \leftarrow M_l[0 : n_l)$, $M_r \leftarrow M_r[0 : n_r)$
$C_l^{\mathsf{sub}} \leftarrow$ FFOSCCTRBITS(M_l, n_l, m, k_l)
$C_r^{\mathsf{sub}} \leftarrow$ FFOSCCTRBITS(M_r, n_r, m, k_r)
return $C_l \parallel C_r \parallel C_l^{\mathsf{sub}} \parallel C_r^{\mathsf{sub}}$

is sufficient because the k_v descendant samples contain at most mk_v distinct selected positions.

Repeating this process recursively converts the global membership decisions in M into the local compaction controls required by all internal nodes. These controls are stored in a fixed traversal order determined only by the public sampling tree; their values remain secret. Once the control array has been generated, the membership information is no longer needed online. FFOSCAPPLY simply consumes the precomputed controls in the same tree order and applies them to the actual records, as described in Section 3.3.

3.3. Online Sampling Execution

Once the payload records become available, FFOSCAPPLY (Algorithm 4) replays the controls generated offline on the actual data. It traverses the same public sampling tree as FFOSCCTRBITS and, at each internal node, applies the corresponding selection flags to independent copies of the current candidate array using oblivious compaction. Each child retains its predetermined public-length prefix and continues recursively. Because the offline phase generated descendant controls using the same sequence of compactions, these controls refer to exactly the record positions encountered online. No membership information is accessed during this phase; all sampling-dependent decisions have already been encoded in C. When a leaf is reached, its m records directly constitute the corresponding sample.

Example. Figure 1 illustrates the complete process for $n = 8$, $m = 2$, and $k = 4$. The membership words shown at the root encode the four sample sets. For example, $M[4] = 1111$ means that D_4 belongs to all four samples and must therefore propagate to both root subtrees, whereas $M[5] = 0000$ means that D_5 is irrelevant to all samples.

At the root, the four samples are divided into the left group (S_0, S_1) and the right group (S_2, S_3). To construct the left child, FFOS-C sets the control bit of position i to 1 iff D_i is selected by at least one sample in (S_0, S_1), i.e., iff either of the first two bits of $M[i]$ is 1. For example, $M[0] = 1000$ gives control bit 1, $M[2] = 0010$ gives 0, and $M[4] = 1111$ gives 1. Applying this rule to all eight positions gives $C_l = (1, 1, 0, 0, 1, 0, 0, 0)$, which selects D_0, D_1, D_4. The left child has public capacity $n_l = \min\{8, 2 \cdot 2\} = 4$. After offline compaction, its retained membership words are $(1000, 0100, 1111, *)$, corresponding online to the candidate array $(D_0, D_1, D_4, *)$. The right child is formed analogously using $C_r = (0, 0, 1, 1, 1, 0, 0, 0)$, retaining $(0010, 0001, 1111, *)$ and the corresponding candidate array $(D_2, D_3, D_4, *)$.

At the left child, the two samples are separated. The first membership bit gives the control for S_0 as $(1, 0, 1, 0)$, while the second gives the control for S_1 as $(0, 1, 1, 0)$. Replaying these controls online produces $S_0 = \{D_0, D_4\}$ and $S_1 = \{D_1, D_4\}$. The right subtree is processed analogously, producing $S_2 = \{D_2, D_4\}$ and $S_3 = \{D_3, D_4\}$.

Algorithm 4 FFOSCAPPLY(D, C, n, m, k, p)

Input: Data array D of length n, control array C, public sample size m, number of samples k, and current position p in C;
Output: Sample list $\mathcal{S}$ and next read position p;

if $k = 1$ **then**
    Copy $D[0 : m)$ into S.
    return $([S], p)$
$k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$
$C_l \leftarrow C[p : p + n)$, $p \leftarrow p + n$ ▷ *left control view*
$C_r \leftarrow C[p : p + n)$, $p \leftarrow p + n$ ▷ *right control view*
$n_l \leftarrow \min(n, mk_l)$, $n_r \leftarrow \min(n, mk_r)$
Copy D into independent arrays D_l and D_r.
OCOMPACT(D_l, C_l)
OCOMPACT(D_r, C_r)
$D_l \leftarrow D_l[0 : n_l)$, $D_r \leftarrow D_r[0 : n_r)$
$(\mathcal{S}_l, p) \leftarrow$ FFOSCAPPLY(D_l, C, n_l, m, k_l, p)
$(\mathcal{S}_r, p) \leftarrow$ FFOSCAPPLY(D_r, C, n_r, m, k_r, p)
return $(\mathcal{S}_l \parallel \mathcal{S}_r, p)$

3.4. Efficiency Optimizations

The public sampling-tree structure also allows the control storage to be determined before control generation begins. Since the tree topology, candidate-array capacities, and the number of selection bits required at every node depend only on the public parameters n, m, and k, the total control-array length can be computed in advance. FFOS-C can therefore allocate a single contiguous array C and assign each subtree a public, disjoint segment of this array. Control generation writes directly into these segments, and

Algorithm 5 BUILDFRONTIER(n, m, k)

Input: Public length n, sample size m $(1 \le m < n)$, and number of samples k $(k > 1)$;
Output: Frontier array $F = [(s_v, k_v)]$;

$q \leftarrow \lfloor n/m \rfloor$ ▷ *Maximum samples per frontier node*
$r \leftarrow \lceil k/q \rceil$ ▷ *Minimum frontier nodes required*
$b \leftarrow \lfloor k/r \rfloor$, $s \leftarrow 0$
$t \leftarrow k \bmod r$
Create an empty frontier array F.
for $v \leftarrow 0 \dots r-1$ **do**
  if $v < t$ **then** $k_v \leftarrow b+1$ ▷ *One extra sample*
  else $k_v \leftarrow b$ ▷ *Use the base sample count*
  Append (s, k_v) to F. ▷ *Record group start and size*
  $s \leftarrow s + k_v$
return F

the online phase later reads them in the same fixed traversal order. This avoids repeated allocation and concatenation of intermediate control arrays and improves memory locality during both offline and online execution.

When $mk > n$, the upper levels of the binary sampling tree may perform substantial redundant work. For a node representing k_v samples, its candidate capacity is $n_v = \min\{n, mk_v\}$; hence, whenever $mk_v \ge n$, splitting the sample group does not yet reduce the candidate-array size. Several consecutive upper levels may therefore repeatedly process full-length arrays before any reduction occurs. To bypass these non-shrinking levels, we introduce a public *frontier* from which the binary recursion begins. Let $q = \lfloor n/m \rfloor$ be the maximum number of samples that can fit in one frontier node and $r = \lceil k/q \rceil$ the minimum number of frontier nodes. BUILDFRONTIER (Algorithm 5) partitions the k samples into r consecutive groups of nearly equal sizes, each satisfying $mk_v \le n$. Each frontier group is then processed from the original input array to form a candidate array of public capacity mk_v, after which the ordinary binary recursion proceeds within that group.

The same tree structure also exposes natural parallelism in both phases. Once an internal node has prepared the inputs and control-array segments for its two children, the left and right subtrees are independent and can be processed concurrently. This recursive independence provides parallelism at every level of the sampling tree. The frontier further increases the available coarse-grained parallelism: its $r = |F|$ groups can be processed independently from the outset, while each group subsequently exposes additional parallelism through its own binary recursion. The actual degree of concurrency can therefore be adapted to the available cores, memory capacity, and bandwidth without changing the public execution structure or the resulting samples.

3.5. Analysis

3.5.1. Correctness. Theorem 1 establishes that FFOS-C generates a batch of independent, uniformly random samples without replacement.

Theorem 1. *Given a dataset D of n indexed records and public parameters $1 \le m < n$ and $k > 1$, let $S = (S_0, \dots, S_{k-1}) =$ FFOS-C(D, n, m, k). Then S consists of k independent, uniformly random samples of m distinct records from D, satisfying Definition 1.*

Proof Sketch. For each sample, MARKSAMPLES selects records according to the remaining selection quota and the number of unprocessed records, yielding a uniformly random selection of exactly m distinct records. Independent randomness across samples ensures that their selections are mutually independent. The control-generation and execution procedures apply the same compaction transformations to the membership information and the corresponding records, preserving their positional alignment. At each sampling-tree node, the retained capacity is sufficient to accommodate all records required by its descendant samples. Consequently, each leaf contains exactly the records selected for its corresponding sample, establishing the correctness of FFOS-C. Detailed proof in Appendix B.1.

3.5.2. Obliviousness. Theorem 2 establishes that FFOS-C is fully oblivious, hiding both the input data and sampling decisions from its execution trace.

Theorem 2. *Given a dataset D of n indexed records and public parameters $1 \le m < n$ and $k > 1$,* FFOS-C(D, n, m, k) *is fully oblivious as defined in Definition 2.*

Proof Sketch. MARKSAMPLES scans all input positions for each sample using fixed loop bounds, while sampling decisions are computed through oblivious operations. During control generation, the sampling-tree structure, candidate-array capacities, and control-array layout depend only on public parameters. Secret membership information affects only the values of compaction controls, without changing the executed instructions or memory accesses.

During online execution, the algorithm follows the same public sampling tree and applies the precomputed controls through fully oblivious compaction. Neither the input records nor the secret controls influence the execution trace. Therefore, the complete memory-access and instruction traces can be simulated using only public parameters, independently of the input data and realized sampling decisions, establishing the claim. Detailed proof in Appendix B.2.

3.5.3. Efficiency. We analyze the efficiency of FFOS-C through the aggregate input size of its oblivious compaction operations, which characterizes both selection-control storage and online compaction workload. A straightforward approach generates k samples independently, requiring k full-length compactions that collectively process kn records. In contrast, FFOS-C amortizes compaction across samples by progressively reducing candidate-array sizes along the sampling tree.

Let V_C denote the aggregate input size across all online compactions. When $mk \le n$ and k is a power of two, we obtain

$$V_C = 2n + 2mk(\log_2 k - 1) \le 2n \log_2 k.$$

Thus, FFOS-C reduces the aggregate compaction input size by a factor of at least $k/(2\log_2 k)$ compared with independent sampling. For fixed n and k, smaller samples further reduce the candidate-array sizes at subsequent tree levels, yielding greater savings.

When $mk > n$, the frontier optimization (Algorithm 5) partitions the samples into groups of k_v samples satisfying $mk_v \le n$, allowing the same capacity-based analysis to apply within each sampling subtree. The detailed analysis is provided in Appendix B.3.

In our implementation, we instantiate OCOMPACT with ORCompact [4], which runs in $O(s \log s)$ time for s fixed-size records. Under the above conditions, FFOS-C achieves an online compaction complexity of

$$O\big((n + mk \log k) \log n\big),$$

compared with $O(kn \log n)$ for independent sampling.

4. FFOS-FR: Fork-Routing-Based Fast Fully Oblivious Sampling

4.1. Structural Observation

Our starting point is a structural property of the sampling tree constructed by FFOS-C. Recall that a node v representing k_v samples maintains a candidate array of public length $n_v = \min\{n_{\mathsf{par}}, mk_v\}$, where n_{par} is the length of its parent candidate array. We observe that once a node satisfies $n_v = mk_v$, this equality is preserved throughout its descendant subtree. Specifically, let the two children of v represent k_l and k_r samples, where $k_l + k_r = k_v$. Their candidate capacities are $n_l = \min\{n_v, mk_l\}$, $n_r = \min\{n_v, mk_r\}$. Since $k_l, k_r < k_v$ and $n_v = mk_v$, we have $n_l = mk_l$, $n_r = mk_r$, and therefore $n_v = n_l + n_r$. Hence, once $n_v = mk_v$ holds at some node, every descendant node satisfies the same relation, while the capacity of each parent is exactly equal to the combined capacities of its two children. We refer to such nodes as *exact-fit nodes*.

This capacity relation reveals a redundancy in the basic FFOS-C construction. At a general internal node, FFOS-C creates two copies of the current candidate array and performs two independent oblivious compactions, one for each child. For an exact-fit node, however, the n_v slots already match the total number of slots required by the two children, $n_l + n_r$. From a capacity perspective, the two child candidate arrays can therefore be viewed as a partition of the parent's storage rather than as two independently materialized copies of the parent array. The remaining difficulty is that a record may be needed by both children and must consequently appear once in each child array.

Surprisingly, we observe that exact-fit nodes always contain sufficient irrelevant records to accommodate such duplication. Partition the n_v candidate records into four classes according to their relevance to the two children. Let N_L and N_R denote the numbers of records required only by the left and right child, respectively; let N_B denote the number required by both children; and let $N_\emptyset$ denote the number required by neither child. Then

$$n_v = N_L + N_R + N_B + N_\emptyset.$$

The numbers of distinct records actually required by the left and right children are $w_l = N_L + N_B$, $w_r = N_R + N_B$. Because these records must fit within the corresponding public capacities, $w_l \le n_l$, $w_r \le n_r$. Combining these inequalities with $n_v = n_l + n_r$ gives

$$\begin{aligned} N_\emptyset - N_B &= n_v - (N_L + N_R + 2N_B) \\ &= (n_l - w_l) + (n_r - w_r) \\ &\ge 0. \end{aligned}$$

Thus, $N_\emptyset \ge N_B$.

In other words, the number of records irrelevant to both children is always at least the number of records that must be duplicated across the two children. The unused positions occupied by these irrelevant records therefore provide enough capacity to accommodate every required duplicate without increasing the total array size.

This property becomes particularly useful after applying the frontier optimization from Section 3.4. For each frontier group containing k_v samples, BUILDFRONTIER guarantees $mk_v \le n$, and the corresponding frontier candidate array is materialized with public length $n_v = mk_v$.

Each frontier node is therefore an exact-fit node. By the closure property above, every node in the subsequent binary recursion is also exact-fit. Consequently, once the frontier candidate arrays have been formed, the remainder of the sampling tree repeatedly satisfies the following two properties:

1) the parent capacity equals the combined capacities of its two children, i.e., $n_v = n_l + n_r$;
2) the number of records irrelevant to both children is no smaller than the number of records that must be duplicated, i.e., $N_\emptyset \ge N_B$.

These observations suggest that the two independent compactions used by FFOS-C are unnecessarily general for exact-fit subtrees. Rather than duplicating the entire parent candidate array and processing the two copies independently, it may be possible to redistribute the parent array directly into its left and right output regions, duplicating only the records that are genuinely required by both children. This motivates a more specialized oblivious mechanism for processing exact-fit nodes, which we develop in the following sections.

4.2. OFR: Oblivious Fork Routing

4.2.1. Oblivious Fork (OFork). To support the joint redistribution required at exact-fit nodes, we extend oblivious swap (OSWAP), a basic data-movement primitive used in practical oblivious compaction and routing algorithms [4], [5], [20]. For two records x and y and a secret control bit

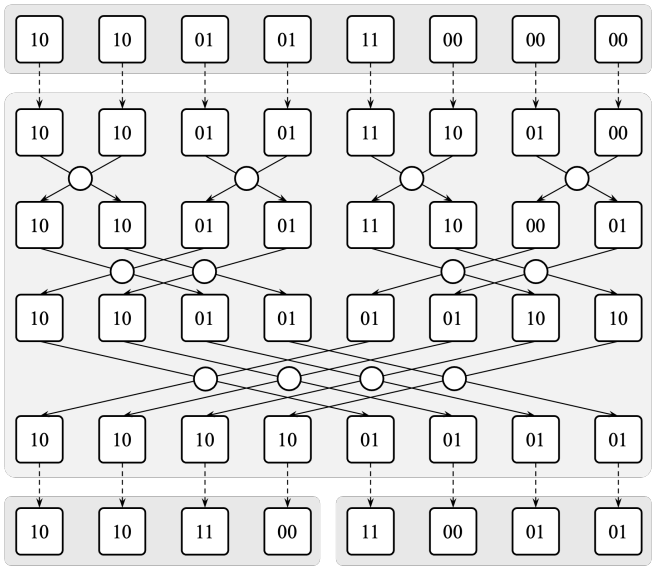


Figure 2. OFR network for $n = 8$ and $n_l = n_r = 4$. Circles denote OFORK gates with fixed butterfly-style connections.

OSWAP

```
movq    (x), %r10
movq    %r10, %r11
movq    (y), %rcx
testb   b, b
cmovnz  %rcx, %r10
cmovnz  %r11, %rcx
movq    %r10, (x)
movq    %rcx, (y)
```

OFORK

```
movq    (x), %r10
movq    %r10, %r11
movq    (y), %rcx
testb   b0, b0
cmovnz  %rcx, %r10
testb   b1, b1
cmovnz  %rcx, %r11
movq    %r10, (x)
movq    %r11, (y)
```

Figure 3. Representative x86-64 assembly-style implementations of OSWAP and OFORK for 8-byte records. Both primitives use two conditional moves and the same number of payload loads and stores.

b, let $\text{OSELECT}(x, y, b)$ return x when $b = 0$ and y when $b = 1$. We can then express OSWAP as

$$\text{OSWAP}(x, y, b) = \big(\text{OSELECT}(x, y, b), \text{OSELECT}(x, y, 1 - b)\big).$$

The complementary controls ensure that the two outputs select different inputs, preserving one occurrence of each input record. To additionally support duplication, we allow the two outputs to select their inputs independently. For a secret control word $c = (b_0, b_1) \in \{0, 1\}^2$, we define *oblivious fork* (OFORK) as

$$\text{OFORK}(x, y, c) = \big(\text{OSELECT}(x, y, b_0), \text{OSELECT}(x, y, b_1)\big).$$

Equivalently,

$$\text{OFORK}(x, y, c) = \begin{cases} (x, x), & c = 00, \\ (x, y), & c = 01, \\ (y, x), & c = 10, \\ (y, y), & c = 11. \end{cases}$$

TABLE 1. EXECUTION LATENCY OF OSWAP AND OFORK, ALL TIMES ARE IN NANOSECONDS (ns).

Operation	Block Size (Bytes)					
	4	**16**	**64**	**256**	**1024**	**4096**
OSwap (Straight)	1.95	3.46	12.80	50.31	258.86	638.03
OSwap (Cross)	1.94	3.54	12.78	50.42	256.30	632.12
OFork (Straight)	2.03	3.45	12.74	49.59	237.66	641.49
OFork (Cross)	2.03	3.53	12.66	49.85	234.16	634.54
OFork (Duplication)	2.04	3.46	12.51	49.67	231.75	643.96

Thus, 01 and 10 implement straight and cross routing, while 00 and 11 duplicate the first and second input, respectively.

This extension preserves the payload-access pattern of OSWAP. Figure 3 shows representative x86-64 implementations for an 8-byte record. Both primitives use two payload loads, two conditional-move selections, and two payload stores. OFORK tests an independent control bit for each output, whereas OSWAP uses a single shared test. Their similar instruction sequences suggest comparable per-operation costs, which we evaluate experimentally in Table 1.

4.2.2. OFR Network. Oblivious Fork Routing (OFR) extends the OFORK operation to the joint redistribution of an entire candidate array. For each input record $D[i]$, we associate a secret two-bit routing tag $T[i] \in \{00, 10, 01, 11\}$, whose first and second bits indicate whether the record is required by the left and right child, respectively. Thus, 10 and 01 request only the left and right output, respectively, 11 requests both, and 00 requests neither. These tags specify the required outputs for each record; they are distinct from the control words that determine individual OFORK selections.

Let the public output capacities be n_l and n_r, with $n = n_l + n_r$ and $\sum_{i=0}^{n-1} T[i][0] \le n_l$, $\sum_{i=0}^{n-1} T[i][1] \le n_r$. OFR rearranges and selectively duplicates the length-n array D so that its first n_l positions form the left output and its remaining n_r positions form the right output. Each record appears once in every output region required by its tag; any remaining output positions are filled with records required by neither child. In particular, a record tagged 11 appears once in each region, using the space provided by irrelevant input records, as established for exact-fit nodes above. The order of records within either output region is unspecified.

We realize this operation using a fixed network of OFORK gates with butterfly-style connections [21]. Figure 2 illustrates an eight-position instance with $n_l = n_r = 4$. Replacing the local swap operation with OFORK retains these fixed connections while enabling selective duplication within the network. The topology, including any bypasses or omitted gates, depends only on the public array length and output capacities; the routing tags affect the secret gate controls.

The network can be evaluated in place on a single length-n data array. Each gate loads its two input records and writes its two outputs back to the same pair of positions. The joint redistribution therefore uses the existing array slots without

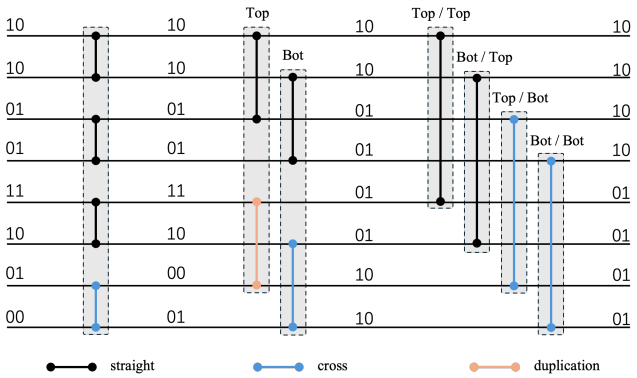


Figure 4. An illustrative responsibility assignment for the OFR network with $n = 8$ and $n_l = n_r = 4$.

TABLE 2. PAIR-ROLE CLASSIFICATION USED BY OFRASSIGN.

Tag pair	o	a_l	a_r	f_l	f_r	Tag pair	o	a_l	a_r	f_l	f_r
$\{00,00\}$	0	0	0	0	0	$\{10,01\}$	**1**	0	0	0	0
$\{00,10\}$	0	**1**	0	0	0	$\{10,11\}$	0	0	**1**	**1**	0
$\{00,01\}$	0	0	**1**	0	0	$\{01,01\}$	0	0	0	0	**1**
$\{00,11\}$	0	**1**	**1**	0	0	$\{01,11\}$	0	**1**	0	0	**1**
$\{10,10\}$	0	0	0	**1**	0	$\{11,11\}$	0	0	0	**1**	**1**

materializing two full-length copies of the candidate array. To obtain the required left and right outputs, however, the controls must coordinate selections across all stages.

4.3. OFR Control Generation and Execution

Algorithm 6 separates OFR into an offline control-generation phase and an online payload-routing phase. The offline phase derives a secret two-bit control word for every OFORK from the routing tags, while the online phase replays these controls on the aligned data array to materialize the left and right outputs. Routing tags and OFORK controls serve different purposes: the former specify which output regions a record must reach, whereas the latter determine how each individual OFORK selects its two outputs.

To bridge these two levels, we interpret each set bit of a routing tag as one *routing responsibility*. Specifically, a tag bit for the left (right) direction represents the obligation to deliver the associated record to one position in the left (right) output region. Thus, tags 10 and 01 carry one left and one right responsibility, respectively, tag 11 carries both, and tag 00 carries neither. The two responsibilities of a 11-tagged record may remain together through intermediate layers and are separated only when the record is duplicated. Control generation therefore amounts to progressively assigning these responsibilities through the OFR network until the desired output tags of every OFORK are determined.

4.3.1. Tag Normalization. The original routing tags encode only the records required by the sampling task and therefore need not account for every position in the two public output regions. In particular, records tagged 00 are irrelevant to both outputs and carry no routing responsibility. Nevertheless, each such record still occupies a position in the fixed OFR network and must eventually be routed to either the left or the right output as an irrelevant filler.

We therefore normalize the routing tags before generating the network controls. OFRNORMALIZE (Algorithm 7) assigns the remaining left and right responsibilities to suitable 00-tagged records while leaving every original nonzero tag unchanged. The exact-fit property established in Section 4.1 guarantees that enough irrelevant records are available for this purpose. After normalization, the tag array satisfies $\sum_i T[i][0] = n_l$, $\sum_i T[i][1] = n_r$.

Hence, the number of left and right responsibilities exactly matches the corresponding output capacities. This exact-count property is the invariant used by the recursive responsibility assignment described next.

4.3.2. Responsibility Assignment. Figure 4 gives an alternative representation of the same OFR network in Figure 2. While Figure 2 emphasizes the fixed butterfly-style wiring, Figure 4 logically groups the wires belonging to the same

Algorithm 6 OFR(D, T, n, n_l, n_r)

Input: Data array D, routing-tag array T of length n, and public parameters n_l and n_r with $n = n_l + n_r$, $\sum_i T[i][0] \le n_l$ and $\sum_i T[i][1] \le n_r$;
Output: Left and right output views (D_l, D_r);

Offline phase
$\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
OFRNORMALIZE(T, n, n_l, n_r) ▷ *Fill deficits*
$C \leftarrow$ OFRCONTROLBITS$(T, \mathcal{V}_0)$
Online phase
$p \leftarrow$ OFRAPPLY$(D, C, \mathcal{V}_0, 0)$ ▷ *Route in place*
$D_l \leftarrow D[0 : n_l)$, $D_r \leftarrow D[n_l : n)$
return (D_l, D_r)

Algorithm 7 OFRNORMALIZE(T, n, n_l, n_r)

Input: Routing-tag array T of length n, and public parameters n, n_l, n_r with $n = n_l + n_r$, $\sum_i T[i][0] \le n_l$ and $\sum_i T[i][1] \le n_r$;
Output: Normalize T in place to exactly n_l left and n_r right responsibilities;

$w_l \leftarrow 0$, $w_r \leftarrow 0$ ▷ *Current left/right responsibilities*
for $i \leftarrow 0 \dots n-1$ **do**
  $w_l \leftarrow w_l + T[i][0]$
  $w_r \leftarrow w_r + T[i][1]$
$\delta_l \leftarrow n_l - w_l$, $\delta_r \leftarrow n_r - w_r$
$\rho_0 \leftarrow 0$ ▷ *Number of preceding 00 tags*
for $i \leftarrow 0 \dots n-1$ **do**
  $z \leftarrow [\![T[i] = 00]\!]$
  $u_l \leftarrow z \wedge [\![\rho_0 < \delta_l]\!]$
  $u_r \leftarrow z \wedge [\![\delta_l \le \rho_0]\!] \wedge [\![\rho_0 < \delta_l + \delta_r]\!]$
  $T[i] \leftarrow$ OSELECT$(T[i], 10, u_l)$
  $T[i] \leftarrow$ OSELECT$(T[i], 01, u_r)$
  $\rho_0 \leftarrow \rho_0 + z$

Algorithm 8 OFRASSIGN$(T, \mathcal{V})$

Input: Routing-tag array T, public recursive view $\mathcal{V} = (a, s, n, n_l, n_r)$.
Output: Current-layer control array C_{cur}, and update T in place to reflect the new routing tags;

▷ *Determine top-subnetwork responsibility targets*
$\ell_g \leftarrow \lfloor n/2 \rfloor$, $n^{\mathsf{top}} \leftarrow \lceil n/2 \rceil$
$n_l^{\mathsf{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\mathsf{top}} \leftarrow n^{\mathsf{top}} - n_l^{\mathsf{top}}$
$b_l \leftarrow 0$, $b_r \leftarrow 0$
if $n \bmod 2 = 1$ **then**
  $b_l \leftarrow T[a + (n-1)s][0]$ ▷ *Odd-size bypass*
  $b_r \leftarrow T[a + (n-1)s][1]$
$\tau_l \leftarrow n_l^{\mathsf{top}} - b_l$, $\tau_r \leftarrow n_r^{\mathsf{top}} - b_r$ ▷ *Paired targets*
▷ *Count pair roles in the current layer*
Initialize counters c_o, c_{a_l}, c_{a_r}, c_{f_l}, and c_{f_r} to 0.
for $i \leftarrow 0 \dots \ell_g - 1$ **do**
  $j_0 \leftarrow a + 2 * i * s$, $j_1 \leftarrow j_0 + s$
  $(o, a_l, a_r, f_l, f_r) \leftarrow$ PAIRROLE$(T[j_0], T[j_1])$
  $c_o \leftarrow c_o + o$
  $c_{a_l} \leftarrow c_{a_l} + a_l$, $c_{a_r} \leftarrow c_{a_r} + a_r$
  $c_{f_l} \leftarrow c_{f_l} + f_l$, $c_{f_r} \leftarrow c_{f_r} + f_r$
▷ *Compute balancing quotas*
$q_o^{\min} \leftarrow \max\{0,\ \tau_l - c_{f_l} - c_{a_l},\ c_{f_r} + c_o - \tau_r\}$
$q_o^{\max} \leftarrow \min\{c_o,\ \tau_l - c_{f_l},\ c_{f_r} + c_o + c_{a_r} - \tau_r\}$
$q_o \leftarrow \min\{q_o^{\max}, \max\{q_o^{\min}, \lfloor c_o/2 \rfloor\}\}$
$q_l \leftarrow \tau_l - c_{f_l} - q_o$
$q_r \leftarrow \tau_r - c_{f_r} - c_o + q_o$
▷ *Assign pairs and generate current-layer controls*
Create a length-ℓ_g two-bit control array C_{cur}.
$\rho_o \leftarrow 0$, $\rho_l \leftarrow 0$, $\rho_r \leftarrow 0$
for $i \leftarrow 0 \dots \ell_g - 1$ **do**
  $j_0 \leftarrow a + 2is$, $j_1 \leftarrow j_0 + s$
  $(o, a_l, a_r, f_l, f_r) \leftarrow$ PAIRROLE$(T[j_0], T[j_1])$
  $\chi_o \leftarrow o \wedge [\![\rho_o < q_o]\!]$
  $\chi_l \leftarrow a_l \wedge [\![\rho_l < q_l]\!]$
  $\chi_r \leftarrow a_r \wedge [\![\rho_r < q_r]\!]$
  $t_l^{\mathsf{top}} \leftarrow f_l \vee \chi_o \vee \chi_l$
  $t_r^{\mathsf{top}} \leftarrow f_r \vee (o \wedge \neg\chi_o) \vee \chi_r$
  $t_l^{\mathsf{bot}} \leftarrow T[j_0][0] + T[j_1][0] - t_l^{\mathsf{top}}$
  $t_r^{\mathsf{bot}} \leftarrow T[j_0][1] + T[j_1][1] - t_r^{\mathsf{top}}$
  $t^{\mathsf{top}} \leftarrow (t_l^{\mathsf{top}}, t_r^{\mathsf{top}})$
  $t^{\mathsf{bot}} \leftarrow (t_l^{\mathsf{bot}}, t_r^{\mathsf{bot}})$
  $C_{\mathsf{cur}}[i] \leftarrow$ OFORKCTRL$(T[j_0], T[j_1], t^{\mathsf{top}}, t^{\mathsf{bot}})$
  $(T[j_0], T[j_1]) \leftarrow (t^{\mathsf{top}}, t^{\mathsf{bot}})$ ▷ *Update tags*
  $\rho_o \leftarrow \rho_o + o$, $\rho_l \leftarrow \rho_l + a_l$, $\rho_r \leftarrow \rho_r + a_r$
return C_{cur}

Algorithm 9 OFRCONTROLBITS$(T, \mathcal{V})$

Input: Routing-tag array T, public recursive view $\mathcal{V} = (a, s, n, n_l, n_r)$;
Output: Complete control array C for OFR network;

$(a, s, n, n_l, n_r) \leftarrow \mathcal{V}$
if $n_l = 0$ **or** $n_r = 0$ **then return** $[\,]$
if $n = 2$ **then**
  $c \leftarrow$ OFORKCTRL$(T[a], T[a+s], 10, 01)$
  $(T[a], T[a+s]) \leftarrow (10, 01)$
  return $[c]$
$n^{\mathsf{top}} \leftarrow \lceil n/2 \rceil$, $n^{\mathsf{bot}} \leftarrow n - n^{\mathsf{top}}$
$n_l^{\mathsf{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\mathsf{top}} \leftarrow n^{\mathsf{top}} - n_l^{\mathsf{top}}$
$n_l^{\mathsf{bot}} \leftarrow n_l - n_l^{\mathsf{top}}$, $n_r^{\mathsf{bot}} \leftarrow n_r - n_r^{\mathsf{top}}$
$\mathcal{V}^{\mathsf{top}} \leftarrow (a, 2s, n^{\mathsf{top}}, n_l^{\mathsf{top}}, n_r^{\mathsf{top}})$ ▷ *Top view*
$\mathcal{V}^{\mathsf{bot}} \leftarrow (a+s, 2s, n^{\mathsf{bot}}, n_l^{\mathsf{bot}}, n_r^{\mathsf{bot}})$ ▷ *Bottom view*
$C_{\mathsf{cur}} \leftarrow$ OFRASSIGN$(T, \mathcal{V})$
$C_{\mathsf{top}} \leftarrow$ OFRCONTROLBITS$(T, \mathcal{V}^{\mathsf{top}})$
$C_{\mathsf{bot}} \leftarrow$ OFRCONTROLBITS$(T, \mathcal{V}^{\mathsf{bot}})$
return $C_{\mathsf{cur}} \,\|\, C_{\mathsf{top}} \,\|\, C_{\mathsf{bot}}$

Algorithm 10 OFORKCTRL$(x, y, t^{\mathsf{top}}, t^{\mathsf{bot}})$

Input: Input tags x, y and target output tags $t^{\mathsf{top}}, t^{\mathsf{bot}}$;
Output: Two-bit control c of OFORK gate;

$z_{\mathsf{straight}} \leftarrow [\![t^{\mathsf{top}} = x]\!] \wedge [\![t^{\mathsf{bot}} = y]\!]$
$z_{\mathsf{cross}} \leftarrow [\![t^{\mathsf{top}} = y]\!] \wedge [\![t^{\mathsf{bot}} = x]\!]$
$z_x \leftarrow [\![x = 11]\!] \wedge [\![y = 00]\!]$
$z_y \leftarrow [\![x = 00]\!] \wedge [\![y = 11]\!]$
$z_{00,11} \leftarrow z_x \vee z_y$
$z_{l,r} \leftarrow [\![t^{\mathsf{top}} = 10]\!] \wedge [\![t^{\mathsf{bot}} = 01]\!]$
$z_{r,l} \leftarrow [\![t^{\mathsf{top}} = 01]\!] \wedge [\![t^{\mathsf{bot}} = 10]\!]$
$z_{\mathsf{dup}} \leftarrow z_{00,11} \wedge (z_{l,r} \vee z_{r,l})$ ▷ *Duplication*
$c_{\mathsf{dup}} \leftarrow (z_y, z_y)$ ▷ *Copy the* 11 *input*
$u_{\mathsf{cross}} \leftarrow z_{\mathsf{cross}} \wedge \neg z_{\mathsf{straight}}$
$u_{\mathsf{dup}} \leftarrow z_{\mathsf{dup}} \wedge \neg z_{\mathsf{straight}} \wedge \neg z_{\mathsf{cross}}$
$c \leftarrow 01$ ▷ *Straight by default*
$c \leftarrow$ OSELECT$(c, 10, u_{\mathsf{cross}})$
$c \leftarrow$ OSELECT$(c, c_{\mathsf{dup}}, u_{\mathsf{dup}})$
return c

recursive subnetwork into shaded regions, making the recursive responsibility assignment explicit. This regrouping is only a logical view of the same network and introduces no additional routing operation.

For every OFORK in the current layer, we regard its left output as entering the *Top* subnetwork and its right output as entering the *Bottom* subnetwork. Top and Bottom describe where an output is processed in the next recursive step, whereas left and right responsibilities describe the final output region that a record must reach. Hence, a left or right responsibility may be assigned to either Top or Bottom at an intermediate layer and continue to be routed within that subnetwork.

The purpose of responsibility assignment is to determine what the two outputs of each OFORK should carry. For an input pair (x, y), we denote the desired output tags by t^{top} and t^{bot}. Once these two tags are fixed, the corresponding two-bit OFORK control can be derived from the input pair and the desired outputs: the gate performs straight routing, cross routing, or duplicates one of its inputs. Thus, control generation is reduced to determining a valid responsibility assignment for the Top and Bottom outputs of every gate.

This assignment is performed recursively. For each current subnetwork, the public network structure determines in advance how many left and right responsibilities its Top and

Algorithm 11 OFRAPPLY$(D, C, \mathcal{V}, p)$

Input: Data array D, control array C, recursive view $\mathcal{V} = (a, s, n, n_l, n_r)$, and read position p;
Output: Next read position p;

$(a, s, n, n_l, n_r) \leftarrow \mathcal{V}$
if $n_l = 0$ **or** $n_r = 0$ **then return** p
if $n = 2$ **then**
    $(x, y) \leftarrow (D[a], D[a + s])$
    $(D[a], D[a + s]) \leftarrow$ OFORK$(x, y, C[p])$
    return $p + 1$
$\ell_g \leftarrow \lfloor n/2 \rfloor$
$n^{\mathsf{top}} \leftarrow \lceil n/2 \rceil$, $n^{\mathsf{bot}} \leftarrow n - n^{\mathsf{top}}$
$n_l^{\mathsf{top}} \leftarrow \lceil n_l/2 \rceil$, $n_r^{\mathsf{top}} \leftarrow n^{\mathsf{top}} - n_l^{\mathsf{top}}$
$n_l^{\mathsf{bot}} \leftarrow n_l - n_l^{\mathsf{top}}$, $n_r^{\mathsf{bot}} \leftarrow n_r - n_r^{\mathsf{top}}$
$\mathcal{V}^{\mathsf{top}} \leftarrow (a, 2s, n^{\mathsf{top}}, n_l^{\mathsf{top}}, n_r^{\mathsf{top}})$
$\mathcal{V}^{\mathsf{bot}} \leftarrow (a + s, 2s, n^{\mathsf{bot}}, n_l^{\mathsf{bot}}, n_r^{\mathsf{bot}})$
for $i \leftarrow 0 \dots \ell_g - 1$ **do**
    $j_0 \leftarrow a + 2 * i * s, \quad j_1 \leftarrow j_0 + s$
    OFORK$(D[j_0], D[j_1], C[p + i])$ ▷ *In-place gate*
$p \leftarrow p + \ell_g$
$p \leftarrow$ OFRAPPLY$(D, C, \mathcal{V}^{\mathsf{top}}, p)$
$p \leftarrow$ OFRAPPLY$(D, C, \mathcal{V}^{\mathsf{bot}}, p)$
return p

Bottom children must receive. The current layer assigns its input responsibilities to satisfy these targets, and the same process is then applied independently to the two children. Figure 4 illustrates this process for $n = 8$ and $n_l = n_r = 4$: after the first layer, the responsibilities are distributed to two smaller subnetworks, each of which is further divided in the next layer. Repeating this process progressively refines subnetwork-level responsibility counts into position-level routing decisions, until the output tags become a contiguous sequence of left-only tags followed by right-only tags.

To determine a valid assignment within one layer, we exploit the fact that each input tag belongs to $\{00, 10, 01, 11\}$. Therefore, a single OFORK has only ten possible unordered input-tag pairs. Table 2 enumerates these cases and classifies them according to the freedom they provide when assigning responsibilities to Top and Bottom. We represent the classification by (o, a_l, a_r, f_l, f_r). Here, f_l and f_r denote left and right responsibilities whose contribution to Top is fixed, while a_l and a_r denote responsibilities that can be assigned to either subnetwork. The indicator o captures the special pair $\{10, 01\}$, for which assigning the left responsibility to Top necessarily assigns the right responsibility to Bottom, or vice versa.

This classification also captures duplication naturally. For example, the pair $\{11, 00\}$ contains one record carrying both responsibilities and one record carrying none. Its two responsibilities may remain together or be separated between Top and Bottom by duplicating the 11-tagged record. In contrast, the two responsibilities in $\{10, 01\}$ belong to different records and therefore form a coupled routing choice. By reducing all ten input cases to these assignment roles, the layer-wide problem becomes selecting how many available choices of each type should contribute to Top.

OFRASSIGN (Algorithm 8) performs this selection. It first determines the responsibility targets of the Top subnetwork, taking into account the unpaired bypass input when the current size is odd. It then scans all input pairs using Table 2 to summarize the fixed and adjustable assignment choices available in the layer. From these counts, OFRASSIGN determines a feasible number of choices of each type that must be assigned to Top. A second fixed-order scan realizes these choices on individual pairs, producing t^{top} and t^{bot} for every OFORK. OFORKCTRL then translates the resulting target tags into the corresponding two-bit control word.

After the current layer is processed, the Top and Bottom subnetworks contain exactly their prescribed numbers of left and right responsibilities. OFRCONTROLBITS (Algorithm 9) therefore applies the same procedure recursively to both children. A subnetwork that requires only one output direction needs no further routing, while a mixed two-position subnetwork is resolved by a single OFORK. Repeating this process determines the desired outputs, and hence the control word, of every OFORK in the OFR network.

4.3.3. Online Apply. Once the controls have been generated, online execution no longer needs the routing tags or performs any responsibility assignment. OFRAPPLY (Algorithm 11) simply replays the precomputed controls on the payload array following the same public recursive order used during control generation: the current layer, followed by the Top and Bottom subnetworks. Accordingly, the control array is stored as $C_{\mathsf{cur}} \parallel C_{\mathsf{top}} \parallel C_{\mathsf{bot}}$. Only the public subnetwork parameters are needed to determine which controls and payload positions are processed at each recursive call.

The responsibility-assignment discussion above logically groups the Top and Bottom subnetworks into separate regions for clarity. OFRAPPLY realizes the same structure without physically separating or merging the payload array. A logical subnetwork is represented by the public view $\mathcal{V} = (a, s, n, n_l, n_r)$, where its j-th logical position is stored at $a + js$. After the current layer is applied, the left output of each OFORK belongs to the Top subnetwork and the right output belongs to the Bottom subnetwork. These two subnetworks are therefore represented by strided views with address parameters $(a, 2s)$ and $(a + s, 2s)$, respectively.

Because both views refer to the same backing array, every OFORK can update its publicly determined pair in place, and the recursive execution requires neither auxiliary payload arrays nor an explicit merge step. Thus, the offline phase determines all secret routing decisions, while the online phase consists only of a fixed sequence of OFORK operations over public addresses. Section 4.4 next integrates OFR into the FFOS sampling tree.

4.4. FFOS-FR Implementation

FFOS-FR integrates OFR into the hierarchical sampling framework of FFOS-C. As shown in Algorithm 12, it retains

Algorithm 12 FFOS-FR(D, n, m, k)

Input: Data array D of public length n, sample size m $(1 \leq m < n)$, and number of samples k $(k > 1)$;
Output: Sample list $\mathcal{S}$ of k independent uniform samples;
Offline phase
$M \leftarrow$ MARKSAMPLES(n, m, k)
if $mk > n$ **then** $F \leftarrow$ BUILDFRONTIER(n, m, k)
else $F \leftarrow \perp$
$C \leftarrow$ FFOSFRCTRBITS(M, F, n, m, k)
Online phase
$(\mathcal{S}, p) \leftarrow$ FFOSFRAPPLY$(D, C, F, n, m, k, (0, 0))$
return $\mathcal{S}$

the same membership generation and sampling-tree structure, but changes how candidate arrays are split once their capacities become exact-fit.

The key is that the frontier group forms a natural transition point between compaction and OFR. Each frontier group containing k_v samples is materialized with public length mk_v and is therefore an exact-fit node. As established in Section 4.1, the exact-fit property is preserved by every descendant. Consequently, after the frontier candidate arrays have been formed, all subsequent internal nodes can be processed by OFR. Compaction is therefore needed only to materialize the initial exact-fit candidate groups, rather than at every level of the sampling tree.

This gives FFOS-FR a simple two-stage structure. Before entering an exact-fit subtree, the required candidate group is materialized using the same oblivious-compaction mechanism as FFOS-C. If $mk > n$, BUILDFRONTIER partitions the samples into several such groups; if $mk < n$, a single candidate array of length mk is materialized. When $mk = n$, the root is already exact-fit and this materialization step is unnecessary. From that point onward, every internal node uses a single OFR instance to jointly produce its two child regions, replacing the two independent compactions required by FFOS-C.

Offline, FFOS-FR derives the routing tags of each exact-fit node from the membership words, generates the corresponding OFR controls, and replays them on the membership array so that the offline membership state remains positionally aligned with the future online payload state. Online execution follows the same public structure: any initial candidate-group materialization is performed using the precomputed compaction controls, after which the remaining recursive subtree is processed entirely by OFRAPPLY.

Thus, the benefit of FFOS-FR is not limited to isolated exact-fit nodes. Once the execution reaches the frontier, OFR replaces the two-compaction split throughout the entire remaining sampling tree. Detailed control generation and online execution are provided in Appendix A.

4.5. Analysis

4.5.1. Correctness.

4.5.2. Obliviousness.

TABLE 3. EXECUTION TIME OF OFR AND ORCOMPACT WITH $n = 2^{20}$, ALL TIMES ARE IN MILLISECONDS (ms).

Operation	Block Size (Bytes)				
	16	**64**	**256**	**1024**	**4096**
ORCompact	24.19	74.68	282.32	1229.80	5104.19
OFR (Online)	23.29	72.93	280.23	1227.74	5090.94
OFR (Offline)	155.93	155.76	158.28	156.71	156.74
Speedup (online)	1.04×	1.02×	1.01×	1.00×	1.00×

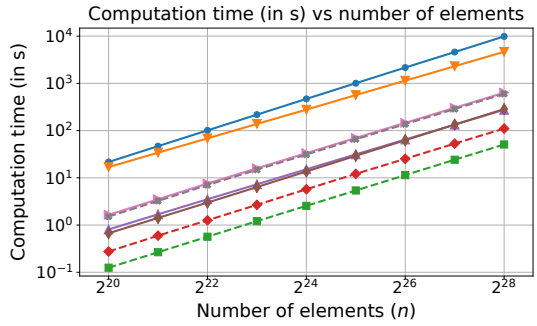
4.5.3. Efficiency.

5. Experiments

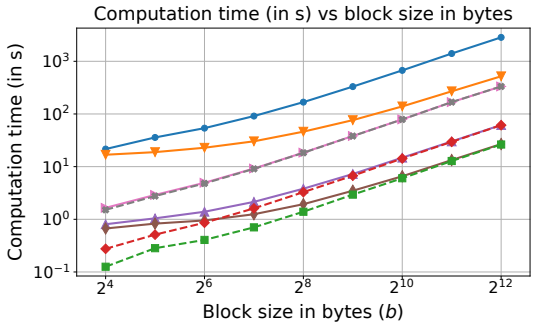
6. Conclusion

Acknowledgments

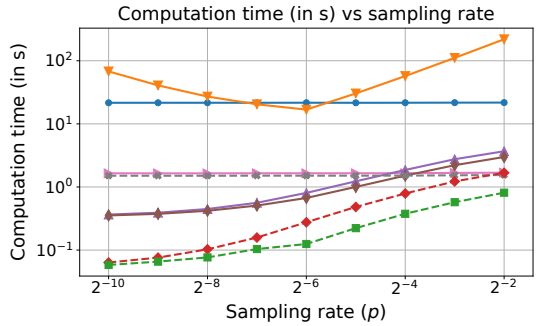
The authors would like to thank...



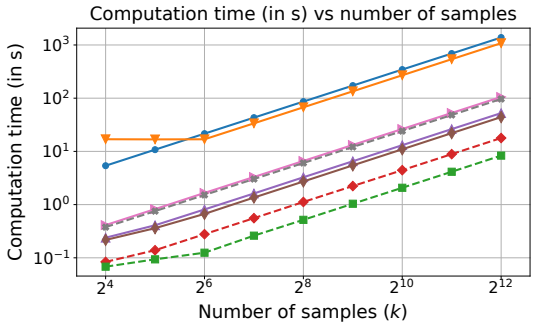
(a) Computation time (in s) with $b = 16\,\mathrm{B},\ p = 1/64,\ k = 64$



(b) Computation time (in s) with $n = 2^{20},\ p = 1/64,\ k = 64$



(c) Computation time (in s) with $n = 2^{20},\ b = 16\,\mathrm{B},\ k = 64$



(d) Computation time (in s) with $n = 2^{20},\ b = 16\,\mathrm{B},\ p = 1/64$



Figure 5. Runtime comparison as (a) input size n, (b) block size b in bytes, (c) sampling rate p, and (d) number of independent sample sets k vary. The default workload uses $n = 2^{20}$, $k = 64$, $p = 1/64$, and $b = 16$ bytes.

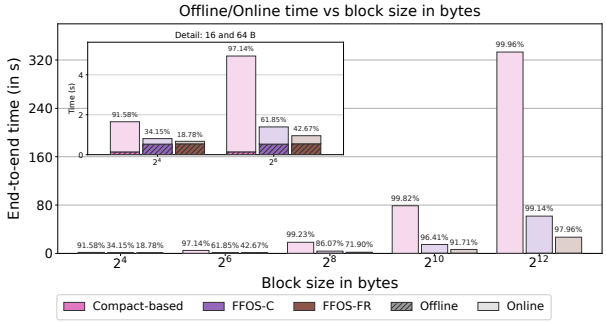


Figure 6. Offline/online runtime of Compact-based, FFOS-C, and FFOS-FR as block size b varies, with $n = 2^{20}$, $k = 64$, and $p = 1/64$. Bar heights show total time in seconds; hatched and light segments show offline and online time, respectively. Labels above the bars give the online percentage of total time.

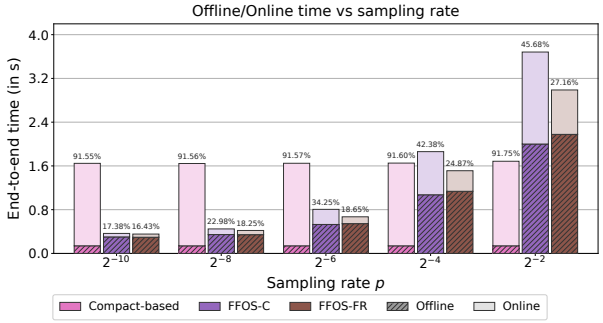


Figure 7. Offline/online runtime of Compact-based, FFOS-C, and FFOS-FR as sampling rate p varies, with $n = 2^{20}$, $k = 64$, and $b = 16$ bytes. Bar heights show total time in seconds; hatched and light segments show offline and online time, respectively. Labels above the bars give the online percentage of total time.

References

[1] X. S. Wang, K. Nayak, C. Liu *et al.*, "Oblivious data structures," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014.

[2] O. Ohrimenko, F. Schuster, C. Fournet *et al.*, "Oblivious multi-party machine learning on trusted processors," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016.

[3] E. Shi, "Path oblivious heap: Optimal and practical oblivious priority queue," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020.

[4] S. Sasy, A. Johnson, and I. Goldberg, "Fast Fully Oblivious Compaction and Shuffling," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.

[5] ——, "Waks-On/Waks-Off: Fast Oblivious Offline/Online Shuffling and Sorting with Waksman Networks," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.

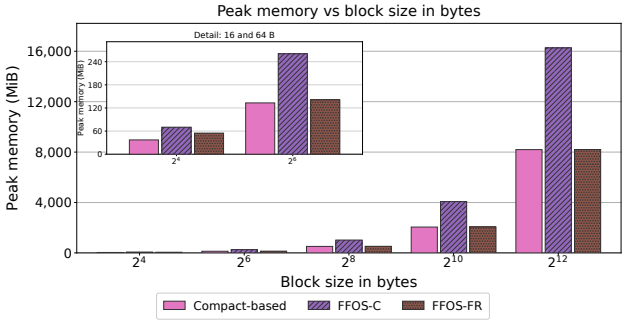


Figure 8. Peak algorithm heap memory (MiB) of Compact-based, FFOS-C, and FFOS-FR as block size b varies, with $n = 2^{20}$, $k = 64$, and $p = 1/64$. The inset expands the 16-byte and 64-byte cases.

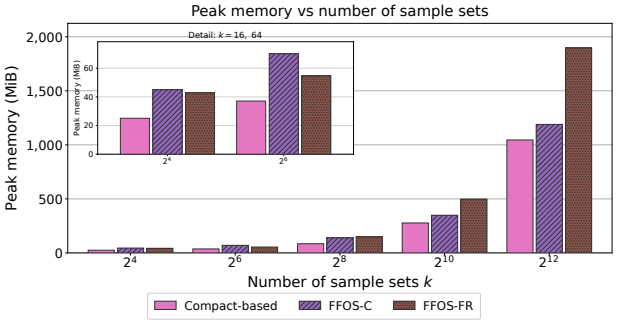


Figure 9. Peak algorithm heap memory (MiB) of Compact-based, FFOS-C, and FFOS-FR as the number of independent sample sets k varies, with $n = 2^{20}$, $p = 1/64$, and $b = 16$ bytes. The inset expands the $k = 16$ and $k = 64$ cases.

[6] T. Gu, Y. Wang, A. Tinoco *et al.*, "Flexway O-Sort: Enclave-friendly and optimal oblivious sorting," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[7] Z. Xu, S. Lai, X. Liu *et al.*, "OblivGNN: Oblivious inference on transductive and inductive graph neural network," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.

[8] J. G. Chamani, I. Demertzis, D. Papadopoulos *et al.*, "GraphOS: Towards oblivious graph processing," *Proceedings of the VLDB Endowment*, 2023.

[9] M. Soleimani, G. Jia, and A. Khandelwal, "Weave: Efficient and expressive oblivious analytics at scale," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025.

[10] T. Murakami, Y. Sei, and R. Eriguchi, "Fully oblivious differential privacy for frequency estimation in the augmented shuffle model with trusted processors," in *35th USENIX Security Symposium (USENIX Security 26)*, 2026.

[11] V. Costan and S. Devadas, "Intel SGX explained," *IACR Cryptology ePrint Archive*, 2016.

[12] S. Sasy and O. Ohrimenko, "Oblivious Sampling Algorithms for Private Data Analysis," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[13] Q. Luo, Y. Wang, K. Yi *et al.*, "Secure Sampling for Approximate Multi-party Query Processing," *Proceedings of the ACM on Management of Data*, 2023.

[14] K. Talwar, S. Wang, A. McMillan *et al.*, "Samplable Anonymous Aggregation for Private Federated Data Analysis," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.

[15] L. Yu, L. Liu, C. Pu *et al.*, "Differentially private model publishing for deep learning," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019.

[16] Z. Wang, R. Zhu, D. Zhou *et al.*, "Sharpness-Aware initialization: Improving differentially private machine learning from first principles," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[17] J. Fang and K. Yi, "Privacy amplification by sampling under user-level differential privacy," *Proceedings of the ACM on Management of Data*, 2024.

[18] À. Miranda-Pascual, J. Parra-Arnau, and T. Strufe, "The adverse effects of omitting records in differential privacy: How sampling and suppression degrade the privacy–utility tradeoff," in *35th USENIX Security Symposium (USENIX Security 26)*, 2026.

[19] S. L. Lohr, *Sampling: Design and Analysis*. Chapman and Hall/CRC, 2021.

[20] M. T. Goodrich, "Data-Oblivious External-Memory Algorithms for the Compaction, Selection, and Sorting of Outsourced Data," in *Proceedings of the Twenty-Third Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2011.

[21] G. Asharov, T.-H. H. Chan, K. Nayak *et al.*, "Bucket Oblivious Sort: An Extremely Simple Oblivious Sort," in *Symposium on Simplicity in Algorithms (SOSA 2020)*, 2020.

[22] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious RAMs," *Journal of the ACM*, 1996.

[23] E. Stefanov, M. van Dijk, E. Shi *et al.*, "Path ORAM: An extremely simple oblivious RAM protocol," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2013.

[24] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *2015 IEEE Symposium on Security and Privacy (SP)*, 2015.

[25] J. Van Bulck, N. Weichbrodt, R. Kapitza *et al.*, "Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[26] S. Lee, M.-W. Shih, P. Gera *et al.*, "Inferring fine-grained control flow inside SGX enclaves with branch shadowing," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[27] S. Sasy, S. Gorbunov, and C. W. Fletcher, "ZeroTrace: Oblivious memory primitives from intel SGX," in *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*, 2018.

[28] A. Tinoco, S. Gao, and E. Shi, "EnigMap: External-memory oblivious map for secure enclaves," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[29] K. E. Batcher, "Sorting Networks and Their Applications," in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1968.

[30] A. Waksman, "A Permutation Network," *Journal of the ACM*, 1968.

[31] M. Ajtai, J. Komlós, and E. Szemerédi, "An $O(n \log n)$ Sorting Network," in *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing (STOC)*, 1983.

[32] M. T. Goodrich, "Zig-zag Sort: A Simple Deterministic Data-Oblivious Sorting Algorithm Running in $O(n \log n)$ Time," in *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing (STOC)*, 2014.

[33] O. Ohrimenko, M. T. Goodrich, R. Tamassia *et al.*, "The Melbourne Shuffle: Improving Oblivious Storage in the Cloud," in *Automata, Languages, and Programming: 41st International Colloquium, ICALP 2014, Proceedings, Part II*, 2014.

[34] S. Patel, G. Persiano, and K. Yeo, "CacheShuffle: A Family of Oblivious Shuffles," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.

[35] W. Holland, O. Ohrimenko, and A. Wirth, "Efficient Oblivious Permutation via the Waksman Network," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security (AsiaCCS)*, 2022.

[36] N. Ngai, I. Demertzis, J. G. Chamani *et al.*, "Distributed and scalable oblivious sorting and shuffling," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.

[37] J. S. Vitter, "Random Sampling with a Reservoir," *ACM Transactions on Mathematical Software*, 1985.

[38] Q. Luo, Q. Xu, and C. Yang, "Secure Multi-Party Sampling over Joins," *Proceedings of the VLDB Endowment*, 2026.

[39] S. Chu, D. Zhuo, E. Shi *et al.*, "Differentially Oblivious Database Joins: Overcoming the Worst-Case Curse of Fully Oblivious Algorithms," in *2nd Conference on Information-Theoretic Cryptography (ITC)*, 2021.

[40] E. Baum, S. Buxbaum, N. Mathai *et al.*, "ORQ: Complex Analytics on Private Data with Strong Security Guarantees," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, 2025.

[41] T.-H. H. Chan, K.-M. Chung, B. M. Maggs *et al.*, "Foundations of Differentially Oblivious Algorithms," in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2019.

[42] J. Allen, B. Ding, J. Kulkarni *et al.*, "An Algorithmic Framework For Differentially Private Data Analysis on Trusted Processors," in *Advances in Neural Information Processing Systems*, 2019.

Algorithm 13 FFOSFRCTRBITS(M, F, n, m, k)

Input: Membership array M of length n, exact-fit frontier array F (or $\perp$), public sample size m, and number of samples k;
Output: Control arrays $C = (C_{\mathsf{cmp}}, C_{\mathsf{ofr}})$;

$(C_{\mathsf{cmp}}, C_{\mathsf{ofr}}) \leftarrow ([\,], [\,])$
if $k = 1$ **then return** C
$k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$
▷ *Process an exact-fit node using OFR*
if $n = mk$ **then**
  $n_l \leftarrow mk_l$, $n_r \leftarrow mk_r$
  $\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
  $\mu_l \leftarrow (1 \ll k_l) - 1$, $\mu_r \leftarrow (1 \ll k_r) - 1$
  Create a length-n two-bit routing-tag array T
  for $i \leftarrow 0 \ldots n-1$ **do**
    $T[i][0] \leftarrow [\![(M[i] \,\&\, \mu_l) \neq 0]\!]$
    $T[i][1] \leftarrow [\![((M[i] \gg k_l) \,\&\, \mu_r) \neq 0]\!]$
  OFRNORMALIZE(T, n, n_l, n_r)
  $C_{\mathsf{node}} \leftarrow$ OFRCONTROLBITS$(T, \mathcal{V}_0)$
  Append C_{node} to C_{ofr}
  $p \leftarrow$ OFRAPPLY$(M, C_{\mathsf{node}}, \mathcal{V}_0, 0)$
  $M_l \leftarrow M[0 : n_l)$, $M_r \leftarrow M[n_l : n)$
  for $i \leftarrow 0 \ldots n_l - 1$ **do**
    $M_l[i] \leftarrow M_l[i] \,\&\, \mu_l$
  for $i \leftarrow 0 \ldots n_r - 1$ **do**
    $M_r[i] \leftarrow (M_r[i] \gg k_l) \,\&\, \mu_r$
  for each child u in the order l, r **do**
    $C^{\mathsf{sub}} \leftarrow$ FFOSFRCTRBITS$(M_u, \perp, n_u, m, k_u)$
    Append $C^{\mathsf{sub}}_{\mathsf{cmp}}$, $C^{\mathsf{sub}}_{\mathsf{ofr}}$ to C_{cmp}, C_{ofr}, respectively
  return C
▷ *Materialize exact-fit frontier groups*
$F' \leftarrow F$ if $F \neq \perp$, otherwise $[(0, k)]$
for $v \leftarrow 0 \ldots |F'| - 1$ **do**
  $(s_v, k_v) \leftarrow F'[v]$
  $n_v \leftarrow \min(n, mk_v)$, $\mu_v \leftarrow (1 \ll k_v) - 1$
  Copy M into M_{tmp}
  if $n_v < n$ **then**
    Create a length-n bit array C_v
    for $i \leftarrow 0 \ldots n-1$ **do**
      $C_v[i] \leftarrow [\![((M[i] \gg s_v) \,\&\, \mu_v) \neq 0]\!]$
    Append C_v to C_{cmp}
    OCOMPACT(M_{tmp}, C_v)
  Create a length-n_v membership array M_v
  for $i \leftarrow 0 \ldots n_v - 1$ **do**
    $M_v[i] \leftarrow (M_{\mathsf{tmp}}[i] \gg s_v) \,\&\, \mu_v$
  $C^{\mathsf{sub}} \leftarrow$ FFOSFRCTRBITS$(M_v, \perp, n_v, m, k_v)$
  Append $C^{\mathsf{sub}}_{\mathsf{cmp}}$, $C^{\mathsf{sub}}_{\mathsf{ofr}}$ to C_{cmp}, C_{ofr}, respectively
return C

Algorithm 14 FFOSFRAPPLY(D, C, F, n, m, k, p)

Input: Data array D of length n, control arrays C, optional exact-fit frontier F (or $\perp$), public sample size m, number of samples k, and current read positions p;
Output: Sample list $\mathcal{S}$ and next read positions p;

$(C_{\mathsf{cmp}}, C_{\mathsf{ofr}}) \leftarrow C$, $(p_{\mathsf{cmp}}, p_{\mathsf{ofr}}) \leftarrow p$, $\mathcal{S} \leftarrow [\,]$
if $k = 1$ **then**
  Copy $D[0 : m)$ into S
  return $([S], p)$
$k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$
▷ *Process an exact-fit node using OFR*
if $n = mk$ **then**
  $n_l \leftarrow mk_l$, $n_r \leftarrow mk_r$
  $\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
  $p_{\mathsf{ofr}} \leftarrow$ OFRAPPLY$(D, C_{\mathsf{ofr}}, \mathcal{V}_0, p_{\mathsf{ofr}})$
  $p \leftarrow (p_{\mathsf{cmp}}, p_{\mathsf{ofr}})$
  $D_l \leftarrow D[0 : n_l)$, $D_r \leftarrow D[n_l : n)$
  $(\mathcal{S}_l, p) \leftarrow$ FFOSFRAPPLY$(D_l, C, \perp, n_l, m, k_l, p)$
  $(\mathcal{S}_r, p) \leftarrow$ FFOSFRAPPLY$(D_r, C, \perp, n_r, m, k_r, p)$
  return $(\mathcal{S}_l \parallel \mathcal{S}_r, p)$
▷ *Materialize and process frontier groups*
if $F \neq \perp$ **then** $F' \leftarrow F$
else $F' \leftarrow [(0, k)]$
for $v \leftarrow 0 \ldots |F'| - 1$ **do**
  $(s_v, k_v) \leftarrow F'[v]$
  $n_v \leftarrow \min(n, mk_v)$
  Copy D into D_{tmp}
  $(p_{\mathsf{cmp}}, p_{\mathsf{ofr}}) \leftarrow p$
  if $n_v < n$ **then**
    $C_v \leftarrow C_{\mathsf{cmp}}[p_{\mathsf{cmp}} : p_{\mathsf{cmp}} + n)$
    $p_{\mathsf{cmp}} \leftarrow p_{\mathsf{cmp}} + n$
    $p \leftarrow (p_{\mathsf{cmp}}, p_{\mathsf{ofr}})$
    OCOMPACT(D_{tmp}, C_v)
  $D_v \leftarrow D_{\mathsf{tmp}}[0 : n_v)$
  $(\mathcal{S}_v, p) \leftarrow$ FFOSFRAPPLY$(D_v, C, \perp, n_v, m, k_v, p)$
  $\mathcal{S} \leftarrow \mathcal{S} \parallel \mathcal{S}_v$
return $(\mathcal{S}, p)$

Appendix

1. Residual Algorithm

2. Detailed Analysis

2.1. Correctness of FFOS-C.

2.2. Full Obliviousness of FFOS-C.

2.3. Efficiency of FFOS-C. We analyze the aggregate input size of the compaction operations performed by FFOS-C. Each compaction on an array of length s generates s one-bit selection controls offline and processes s records online. Hence, the aggregate compaction input size V_C also equals the number of stored selection-control bits.

We first consider the general case with frontier optimization. Let $\mathcal{F}$ denote the initial sample groups, where group v contains k_v samples, with

$$\sum_{v \in \mathcal{F}} k_v = k \quad \text{and} \quad mk_v \le n.$$

When $mk \le n$, these are the two groups formed by the initial binary split; otherwise, they are constructed by Algorithm 5. Let c be the number of groups satisfying $mk_v < n$, each requiring one initial compaction over n records.

After this initial step, each group enters an exact-fit sampling subtree, where a node representing t samples has capacity mt. Let $L(t)$ denote the sum of the sample counts over all internal nodes of such a subtree. Since the tree recursively splits its samples into two nearly equal groups,

$$L(t) = \begin{cases} 0, & t = 1, \\ t + L(\lfloor t/2 \rfloor) + L(\lceil t/2 \rceil), & t > 1. \end{cases}$$

Each internal node performs two compactions over its candidate array. Therefore, the total compaction input size is

$$V_C = cn + 2m \sum_{v \in \mathcal{F}} L(k_v).$$

When $mk \le n$ and $k = 2^h$, the initial split produces two groups of $k/2$ samples, each requiring a compaction over n records. Since $L(2^j) = 2^j j$, we obtain

$$\begin{aligned} V_C &= 2n + 2m \left(2 \cdot \frac{k}{2} \log_2 \frac{k}{2} \right) \\ &= 2n + 2mk(\log_2 k - 1) \\ &\le 2n \log_2 k, \end{aligned}$$

where the last inequality follows from $mk \le n$. In comparison, independently generating k samples requires compactions with an aggregate input size of kn. Thus, FFOS-C reduces this quantity by a factor of at least $k/(2 \log_2 k)$.

For $mk > n$, the frontier optimization sets $q = \lfloor n/m \rfloor$ and creates $r = \lceil k/q \rceil$ groups, each satisfying $k_v \le q$. Since $L(t) \le t \lceil \log_2 t \rceil$, we have $L(k_v) \le k_v \lceil \log_2 q \rceil$. Together with $c \le r$ and $\sum_v k_v = k$, this yields

$$\begin{aligned} V_C &= cn + 2m \sum_{v \in \mathcal{F}} L(k_v) \\ &\le rn + 2m \sum_{v \in \mathcal{F}} k_v \lceil \log_2 q \rceil \\ &= rn + 2mk \lceil \log_2 q \rceil. \end{aligned}$$

Thus, the frontier bounds the recursive compaction workload by the capacity of each group, avoiding redundant full-length compactions at higher tree levels.

Finally, ORCompact [4] runs in $O(s \log s)$ time on s fixed-size records. Since every compaction input has length at most n, its aggregate online cost is $O(V_C \log n)$. In particular, when $mk \le n$ and k is a power of two, this gives

$$O\big((n + mk \log k) \log n\big),$$

compared with $O(kn \log n)$ for independent sampling.

2.4. Correctness of OFR.

2.5. Correctness of FFOS-FR.

2.6. Full Obliviousness of FFOS-FR.

2.7. Online Gate Reduction.

3. Detailed Axperiments

Figures 10 and 11 report the offline/online runtime breakdown as data size n and the number of independent sample sets k vary. Figures 12 and 13 show peak algorithm heap memory as n and sampling rate p vary.

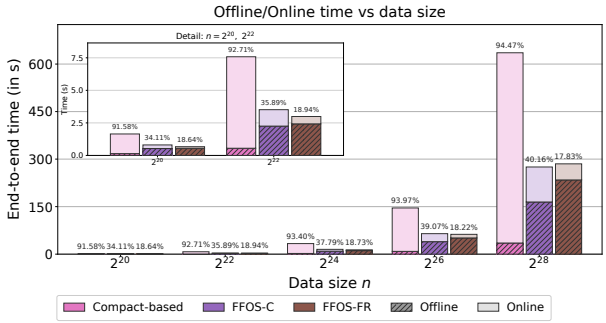


Figure 10. Offline/online runtime of Compact-based, FFOS-C, and FFOS-FR as data size n varies, with $k = 64$, $p = 1/64$, and $b = 16$ bytes. Bar heights show total time in seconds; hatched and light segments show offline and online time, respectively. Labels above the bars give the online percentage of total time. The inset expands the $n = 2^{20}$ and $n = 2^{22}$ cases.

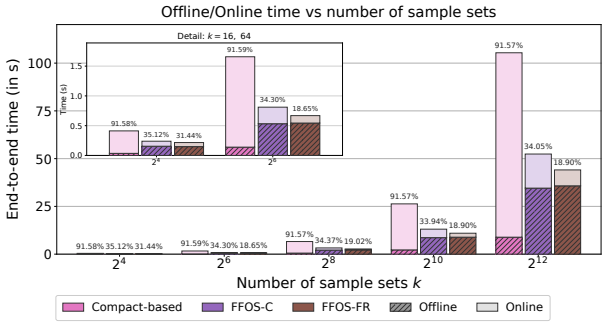


Figure 11. Offline/online runtime of Compact-based, FFOS-C, and FFOS-FR as the number of independent sample sets k varies, with $n = 2^{20}$, $p = 1/64$, and $b = 16$ bytes. Bar heights show total time in seconds; hatched and light segments show offline and online time, respectively. Labels above the bars give the online percentage of total time. The inset expands the $k = 16$ and $k = 64$ cases.

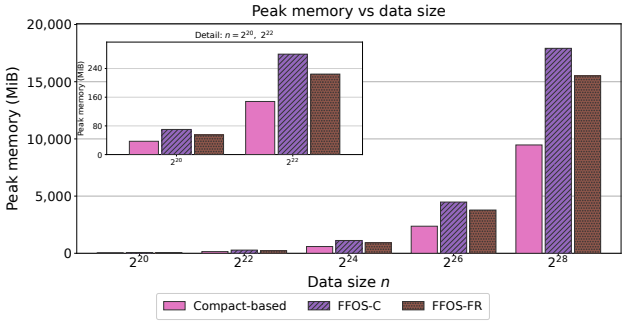


Figure 12. Peak algorithm heap memory (MiB) of Compact-based, FFOS-C, and FFOS-FR as data size n varies, with $k = 64$, $p = 1/64$, and $b = 16$ bytes. The inset expands the $n = 2^{20}$ and $n = 2^{22}$ cases.

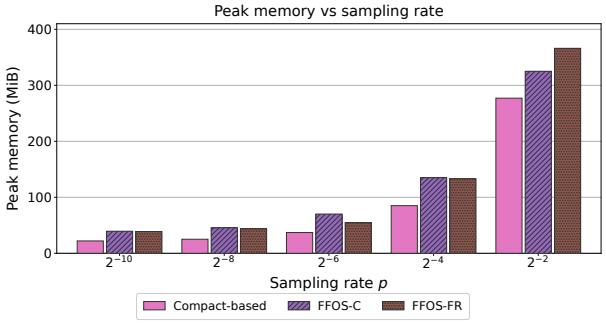


Figure 13. Peak algorithm heap memory (MiB) of Compact-based, FFOS-C, and FFOS-FR as sampling rate p varies, with $n = 2^{20}$, $k = 64$, and $b = 16$ bytes.